

Syrian Arab Republic

Ministry of Higher Education

Syrian Virtual University



Bachelor of IT Program BAIT

MS SQL Server Administration IIS404

Chapter five

Performance Techniques in MS SQL Server

Eng. Ayham Mohammad

Contents

1.	Indexes.....	1
1.1.	B-Tree	1
1.2.	Clustered Index.....	2
1.2.1.	Clustered Index Selection	2
1.3.	Non-Clustered Index.....	3
1.3.1.	Using Non-Clustered Index.....	3
1.3.2.	Non- Clustered Index - Included Columns	3
1.4.	Indexes and Constraints	4
1.5.	Columnstore indexes.....	4
1.5.1.	Batch-mode query processing	5
2.	Partitioning	6
2.1.	Benefits of Partitioning	7
2.2.	Components and Concepts	8
2.2.1.	Partition function	8
2.2.2.	Partition scheme	8
2.2.3.	Partitioning column	8
2.3.	Partitioning Steps.....	8
3.	Full-text Search.....	9
3.1.	Full-text Queries	9
3.1.1.	Set up full-text search steps.....	10
3.1.2.	Full-Text Search Vs LIKE	10
4.	Buffer Pool Extension	11
4.1.	Terms and concepts	11
5.	Memory optimized table.....	12
5.1.	Indexes on memory-optimized tables	12

5.2.	Concurrency Improvements	13
5.3.	Memory optimized table – Hash Index.....	13
5.4.	Creating Memory optimized table	14
5.4.1.	Planning data migration to memory-optimized tables.....	14
5.5.	Natively compiled stored procedures.....	18
5.6.	Querying Memory optimized table.....	19
5.6.1.	Creating Native Stored Procedure	19
5.7.	In-memory OLTP customer success story	20

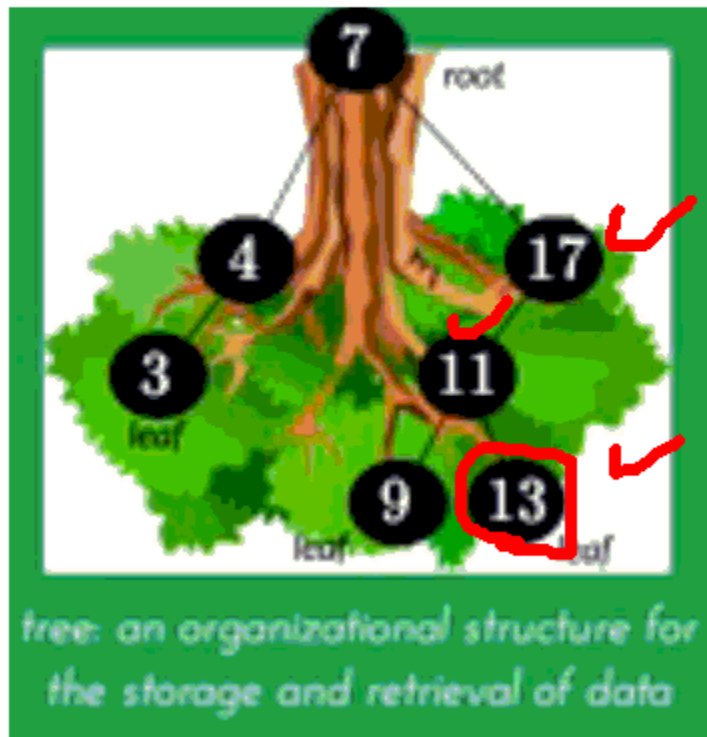
1. Indexes

- الأداء يكون أعلى عن طريق زيادة سرعة الإستجابة مثلاً
- كل حالة أو بيئة عمل لها إستخدام و حل خاص يساعد على زيادة أداء قاعدة البيانات
- ال Index هو الفهرس الذي على أساسه نبحث عن المعلومات (يجعل البحث أسرع من ناحية القراءة و الكتابة) حيث يحتوي ال Index على مفتاح أو مفاتيح مبنية على عامود أو أكثر من جدول أو View.

An index is an on-disk structure associated with a table or a view that speeds retrieval of rows from the table or view. An index contains keys built from one or more columns in the table or view.

يتم حفظ هذه المفاتيح في بنية إسمها B-tree (Balanced tree – أو Binary Search Tree)

B-Tree



التي تسمح لل SQL

Server بالبحث و

العثور على السطور

المرتبطة بهذه المفاتيح

بسرعة أكبر بسبب

إعتمادها على خوارزمية

ال B-tree

التي تقوم على أساس

البداء بنصف القيمة

المراد البحث عنها ثم

الذهاب للأكبر أو الأصغر

إلى أن نصل للقيمة

المرادة

These keys are stored in a structure (B-tree) that enables SQL Server to find the row or rows associated with the key values quickly and efficiently.

ال B-tree تقوم بموازنة نفسها بنفسها عند إضافة أو حذف أي قيمة

يتم في كل أنواع ال B-trees (مثل أيضا ال AVL, Red Black Trees), يتم نسخ كل المفاتيح إلى

الذاكرة لكي نحصل على أداء عالي من قاعدة البيانات

و عندما يكون عدد المفاتيح كبير جدا, أو حجم الداتا كبير جدا, يتم قراءة المفاتيح و البيانات من القرص الصلب

على شكل Blocks

- الفكرة الرئيسية من استخدام ال B-Trees هي تقليل استخدام و الوصول للقرص الصلب

SQL Server has two main types of indexes:

- في ال SQL Server لدينا نوعين من ال Indexes:

- Clustered (Could be unique or non-unique)
- Nonclustered (Could be unique or non-unique)

- لا فرق بين Index منشأ يدويا أو أوتوماتيكيا غير أننا في حال الإنشاء اليدوي نستطيع تحديد إسمه بشكل يناسبنا

- عند تعيين حقل في جدول ك Unique, يتم إنشاء Unique Nonclustered Index بشكل تلقائي له

- عند إنشاء PK يدويا, يتم أوتوماتيكيا إنشاء Clustered Index بشكل تلقائي له

Both clustered and nonclustered indexes can be unique. This means no two rows can have the same value for the index key. Otherwise, the index is not unique and multiple rows can share the same key value.

A unique index guarantees that the index key contains no duplicate values and therefore every row in the table is in some way unique. There are no significant differences between creating a UNIQUE constraint and creating a unique index that is independent of a constraint. Data validation occurs in the same manner, and the query optimizer does not differentiate between a unique index created by a constraint or manually created. However, creating a UNIQUE constraint on the column makes the objective of the index clear.

You cannot create a unique index on a single column if that column contains NULL in more than one row. Similarly, you cannot create a unique index on multiple columns if the combination of columns contains NULL in more than one row. These are treated as duplicate values for indexing purposes

1.1. B-Tree

B-Tree is a self-balancing search tree. In most of the other self-balancing search trees (AVL and Red

Black Trees), it is assumed that everything is in main memory. To understand use of B-Trees, we must think of huge amount of data that cannot fit in main memory.

When the number of keys is high, the data is read from disk in the form of blocks. Disk access time is very high compared to main memory access time. The main idea of using B-Trees is to reduce the number of disk accesses

- الفرق بين ال Clustered Index, Nonclustered Index :

(1) في ال Clustered Index يتم التمييز و الترتيب حسب صنف واحد فقط فيزيائيا, و إنشاءه ليس إجباري.

حيث في حال عدم وجود Clustered Index للجدول يتم ترتيب الأسطر بشكل غير فعال يطلق عليه Heap

و ممكن لل Clustered Index أن يكون Unique, Not Unique

و ممكن في حال كان Unique أن يحتوي Null Value و لكن ليس لأكثر من مرة

- إذا ال Clustered Index هو الحقل (أو مجموعة حقول مرتبة) المفهرس الذي يتم على أساسه الترتيب

فيزيائيا

- طبعا الأفضل إستخدام ال Clustered Index محقل واحد, و يحقق أفضل النتائج عندما تكون ال Query

عبارة عن Range و ليس إستعلام عن قيمة واحدة

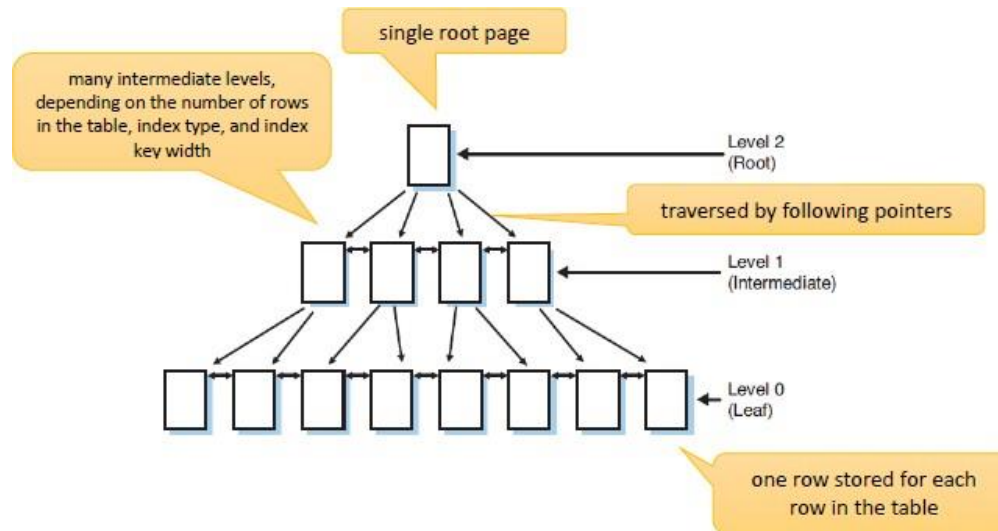
- بينما ال Nonclustered Index يتم به الترتيب بشكل مستقل عن الترتيب الفيزيائي (منفصل عن البنية)

- ممكن إنشاء أكثر من Nonclustered Index

- ممكن الحصول على حالة عدم وجود لأي Nonclustered Index

- يعطينا ال Nonclustered Index أفضل أداء عند إستخدامه للإستعلام عن قيمة ثابتة و ليس Range

- عند إنشاء قيد Unique, يتم إنشاء Nonclustered Unique Index له بشكل أوتوماتيكي



1.2. Clustered Index

Clustered indexes sort and store the data rows in the table or view based on their key values. **There can be only one clustered index per table, because the data rows themselves can be sorted in only one order.**

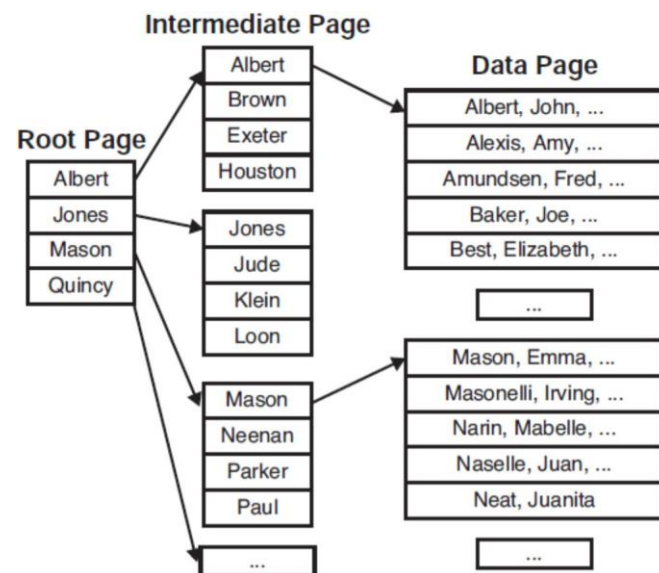
If a table has no clustered index, its data rows are stored in an unordered structure called a heap.

1.2.1. Clustered Index Selection

Should be driven by the way the data is most commonly accessed in the table.

You should consider using the following types of columns in a clustered index:

- Those that are often accessed sequentially
- Those that contain a large number of distinct values



- Those that are used in range queries that use operators such as BETWEEN, >, >=, <, or <= in the WHERE clause
- Those that are frequently used by queries to join or group the result set

1.3. Non-Clustered Index

A nonclustered index is a separate index structure, independent of the physical sort order of the data rows in the table. You are therefore not restricted to creating only one nonclustered index per table.

All index key values are stored in the nonclustered index levels in sorted order, based on the index key(s). The leaf row is independent of the data rows in the table. The leaf level contains a row for every data row in the table, along with a pointer to locate the data row.

This pointer is either

- the clustered index key for the data row, if the table has a clustered index on it
- Or the data page ID and row ID of the data row if the table is stored as a heap structure

1.3.1. Using Non-Clustered Index

SQL Server Profiler and Database Engine Tuning Advisor can help you evaluate your data access paths and determine which columns are the best candidates.

Consider using nonclustered indexes for the following:

- Queries that do not return large result sets
- Columns that are frequently used in the WHERE clause that return exact matches
- Columns that have many distinct values (high cardinality)
- All columns referenced in a critical query

1.3.2. Non-Clustered Index - Included Columns

You could consider using included columns if you have a critical query that often selects last name, first name, and address from a table but uses only the last name and first name as search arguments in the WHERE clause. This may be a situation in which you would want to consider the use of a covering index that places all the referenced columns from the query into a nonclustered index.

In the case of our critical query, the address column can be added to the index as an included column. It is not included in the index key, but it is available in the leaf-level pages of the index so that the additional overhead of going to the data pages to retrieve the address is not needed

- للقيام بإستعلام أسرع بواسطة **Non-Clustered Index** نقوم بوضع قيم الشرط **where** بشكل تعليمية **Include** كما مبين في الصورة التالية:

```
Select LastName, FirstName, Address
From Employees
Where LastName = 'xx' and FirstName = 'yy'

CREATE NONCLUSTERED INDEX enci ON Employees
(
  [LastName] ASC,
  [FirstName] ASC
)
INCLUDE ( [Address])
```

1.4. Indexes and Constraints

مثال:
نقوم بإنشاء DB

Create database svu
On (name='svudata', filename='E:\svu\bait\...\svu.mdf')
Log on (name='svylog', Filename='E:\svu\bait\...\svu.ldf')
Go

Use svu

ثم نقوم بإنشاء **Clustered Index**

Create table TestConstraint (id int not null,
[name] varchar (10) unique, grade int not null)

Create clustered index clix on TestConstraint (grade)

نقوم بإنشاء **PK**, و الذي بشكل افتراضي يتوكد معه **Clustered Index**

Alter table TestConstraint add constraint pk primary key (id)

وفي هذه الحالة يصبح لدينا **2 Clustered Indexes** وهذا أمر خطأ
لذلك سيتم وضع ال **PK** ك **non-clustered Index** لأنه غير مسموح بوجود أكثر من **Clustered Idnex** واحد

Indexes are automatically created when PRIMARY KEY and UNIQUE constraints are defined on

table columns. For example, when you create a table with a UNIQUE constraint, Database Engine automatically creates a non-clustered index. If you configure a PRIMARY KEY, Database Engine automatically creates a clustered index, unless a clustered index already exists. When you try to enforce a PRIMARY KEY constraint on an existing table and a clustered index already exists on that table, SQL Server enforces the primary key using a nonclustered index.

1.5. Columnstore indexes

- تعتبر أحدث من ال B-tree Index أو ال Row Index , يعمل على الذاكرة, و تكتب ال Data مضغوطة 10 مرات على شكل صفحات بناء على الأعمدة وليس الأسطر

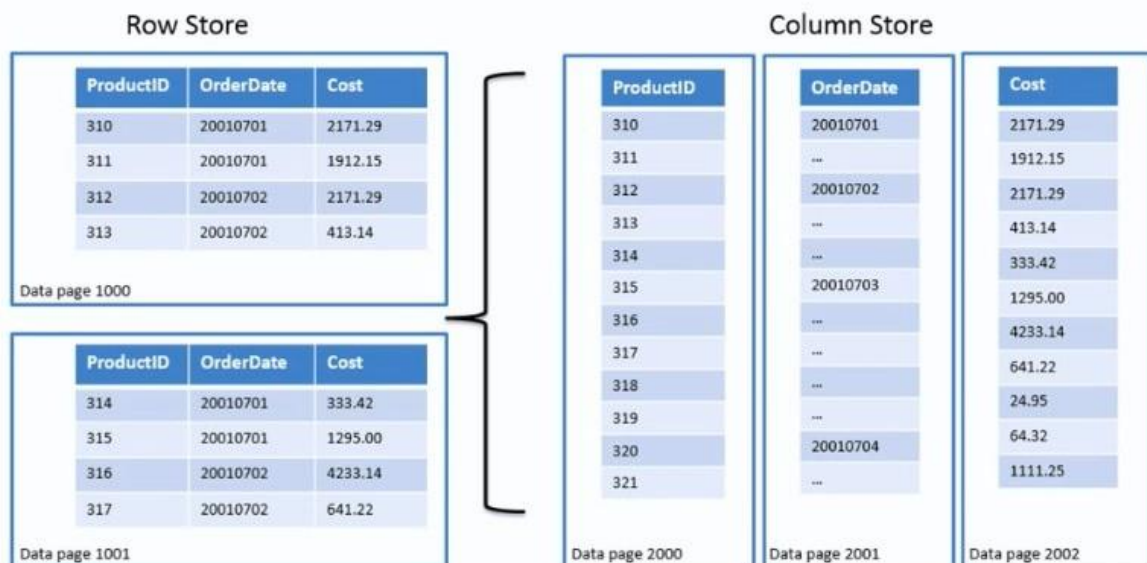
- الإستخدام الأفضل لل Columnstore Index يكون عند إستخدام Fact Clustered Cloumnstore Index على Fact tables, Large Dimension tables في ال data warehousing

(ال Fact Table هو جدول الحقيقة الذي يحتوي جميع المعلومات في ال Data Warehouse , و ال Dimension Tables هي جداول الأبعاد التي تكون حول جدول الحقيقة و تحتوي معلومات معينة منه)

أو إستخدام None-clustered Columnstore Index للقيام ب Analysis في أرض الواقع على OLTP Workload حيث يساعد تحليل البيانات في الحصول على المعلومة التي نبنى عليها القرار (ال OLTP – Online Transaction Processing هي عملية الكتابة و القراءة المستمرة على ال DB – مثل أي برنامج محاسبة أو مبيعات)

و ال OLAP – Online Analysis Processing هي عملية تحليل البيانات بشكل مستمر بناء على حقل معين)

- In-memory, compressed data in pages based on columns instead of rows



Columnstore indexes are the standard for storing and querying large data warehousing fact tables. This index uses column-based data storage and query processing to achieve gains up to 10 times the query performance in data warehouse over traditional row-oriented storage.

It achieves gains up to 10 times the data compression over the uncompressed data size. Use a clustered columnstore index to store fact tables and large dimension tables for data warehousing workloads. This method improves query performance and data compression by up to 10 times. Use a nonclustered columnstore index to perform analysis in real time on an OLTP workload.

Rowstore indexes perform best on queries that seek into the data, when searching for a particular value, or for queries on a small range of values. Use rowstore indexes with transactional workloads because they tend to require mostly table seeks instead of table scans.

Columnstore indexes give high performance gains for analytic queries that scan large amounts of data, especially on large tables. Use columnstore indexes on data warehousing and analytics workloads, especially on fact tables, because they tend to require full table scans rather than table seeks.

- خلاصة:

ينصح باستخدام ال Row Index عندما يكون لدينا Seek لل Data (بحث عن Data إن كانت قيمة واحدة أو نطاق قيم)

و ينصح باستخدام ال Cloumnstore Index عندما يكون لدينا Scan لل Data (Scan = مسح كل الجدول لتنفيذ تابع تجميعي مثلا)

1.5.1. Batch-mode query processing

- طريقة عمل ال Cloumnstore Idnex يطلق عليها Batch-mode Query Processing

و التي تعتمد على خوارزمية Vector-based Query execution , حيث عوضا عن معالجة كل سطر على حدى...

يتم معالجة مجموعة أسطر سوياً مما يعطي أداء أعلى ب 3-4 مرات

- ابتداء من ال 2019 أصبح بإستطاعتنا تفعيل نمط ال Batch-mode Query Processing

- مثال:

نقوم بإنشاء Table جديد:

Create table citable

(id int, theDate date, Note varchar (50))

ثم نقوم بتعبئته ب Data (فعليا حوالي 100000 عملية إدخال):

Declare @id int = 1

While @id <= 100000

Begin

Insert into citable values (@id, getdate() + @id, 'This is the note number #' +

Set @id = @id+2

End

Set @id=2

Declare @date datetime = '20170101'

While @id<=100000

Begin

Insert into citable values (@id, @date+@id, 'This is the note number #' +

Set @id= @id+2

End

Select count(*) from citable

يظهر الناتج = 100000

تعليمات ال into ---- select وظيفتها إنشاء نفس ال Table المستخدم و تعبئته نفس ال Data

Select * into ncitable from citable

Select * into csitable from citable

Select * into ncsitable from citable

نقوم بمقارنة الأداء :

نقوم أولا بتنفيذ تجربة لهذه ال Query عن طريق Test Execution Plan لنعرف ال Execution Time و يكون ناتج ال

Cost = 0.59

Select * from citable where id between 300 and 340 and thedate between '20190101' and '20190525'

تم نقوم بإنشاء Indexes لتجربتها على نفس ال Query ونرى الفرق بال Execution Time :

Create clustered index ci on citable (id,thedata)

Create nonclustered index nci on ncitable (id,thedata)

Create clustered columnstore index csi on csitable (id,thedata)

Create nonclustered cloumnstore index ncsi on ncsitable (id,thedata)

نقوم ب Execute plan للإستعلامات التالية:

Select * from citable where id between 300 and 340 and the date between '20190101' and '20190525'

Select * from ncitable where id between 300 and 340 and the date between '20190101' and '20190525'

Select * from csitable where id between 300 and 340 and the date between '20190101' and '20190525'

Select * from ncsitable where id between 300 and 340 and the date between '20190101' and '20190525'

نرى أن قيمة ال cost = 0.056 أي تحسنت بنسبة 10% بشكل عام (في الإستعلامات عن الجداول المستخدم بها Clustered Index كان التحسن بنسبة 300% تقريبا لأنها تعمل بأفضل شكل عند الإستعلام عن مجال من القيم مثل ما فعلنا)

نقوم بزيادة ال Data الموجودة مرة أخرى:

Insert into citable select * from citable

Insert into ncitable select * from ncitable

Insert into csitable select * from csitable

Insert into ncsitable select * from ncsitable

هذه المرة نقارن عن طريق إستخدام Query تحتوي توابع تحليلية:

نقوم بتنفيذ هذه الإستعلامات عن طريق Execute plane:

Select datepart (month, thedate), avg(id), sum(id), from citable group by

Select datepart (month, thedate), avg(id), sum(id), from ncsitable group by

Select datepart (month, thedate), avg(id), sum(id), from csitable group by

نحصل على قيمة cost = 2.8 عند في الجداول المستخدم فيها Clustered Index

و قيمة $\text{cost} = 0.8$ عند الجداول المستخدم فيها **none clustered index**

و قيمة $\text{cost} = 0.2$ في الجداول المستخدم فيها **Columnstore Index** (أحسن ب 10 مرات من ال **Clustered** عند

إستخدامه مع توابع تجميعية)

- نستطيع أن نقوم بعمليات صيانة لل **Indexes** التي قمنا بإنشائها, حيث كل ال **Indexes** يكون بها **Fragmentation** و نقوم نحن عند صيانتها بعملية **Defragmentation** (إلغاء التجزئة)

و يتم ذلك بإحدى طريقتين:

(1) **Re-Organize** : نستخدمها عند وجود تجزئة بنسبة 30% فما أقل

(2) **Re-Build**: نقوم بإعادة بناء ال **Index** عند وجود تجزئة **Fragmentation** بنسبة تتجاوز ال 30%

نقوم أولا بعرض نسبة ال **Fragmentation** للجدولين **citible, ncitable** كمثال

```
Select avg_fragmentation_in_percent, fragment_count from sys.dm_db_index_physical_stats
(db_id(N'SVU'), object_id('citible'), 1, null, 'limited');
```

```
Select avg_fragmentation_in_percent, fragment_count from sys.dm_db_index_physical_stats
(db_id(N'SVU'), object_id('ncitable'), 2, null, 'limited');
```

تظهر نسبة ال **fragmentation = 99%**

ثم نقوم بتطبيق **Rebuild** عليهم: (سيتم إستخدام طريقة ال **Rebuild** نظرا لأن النسبة تجاوزت ال 30%)

```
Alter index ci on citible rebuild
```

```
Alter index nci on ncitable rebuild
```

نقوم بإعادة قياس نسبة ال **Fragmentation** في كل من الجدولين السابقين:

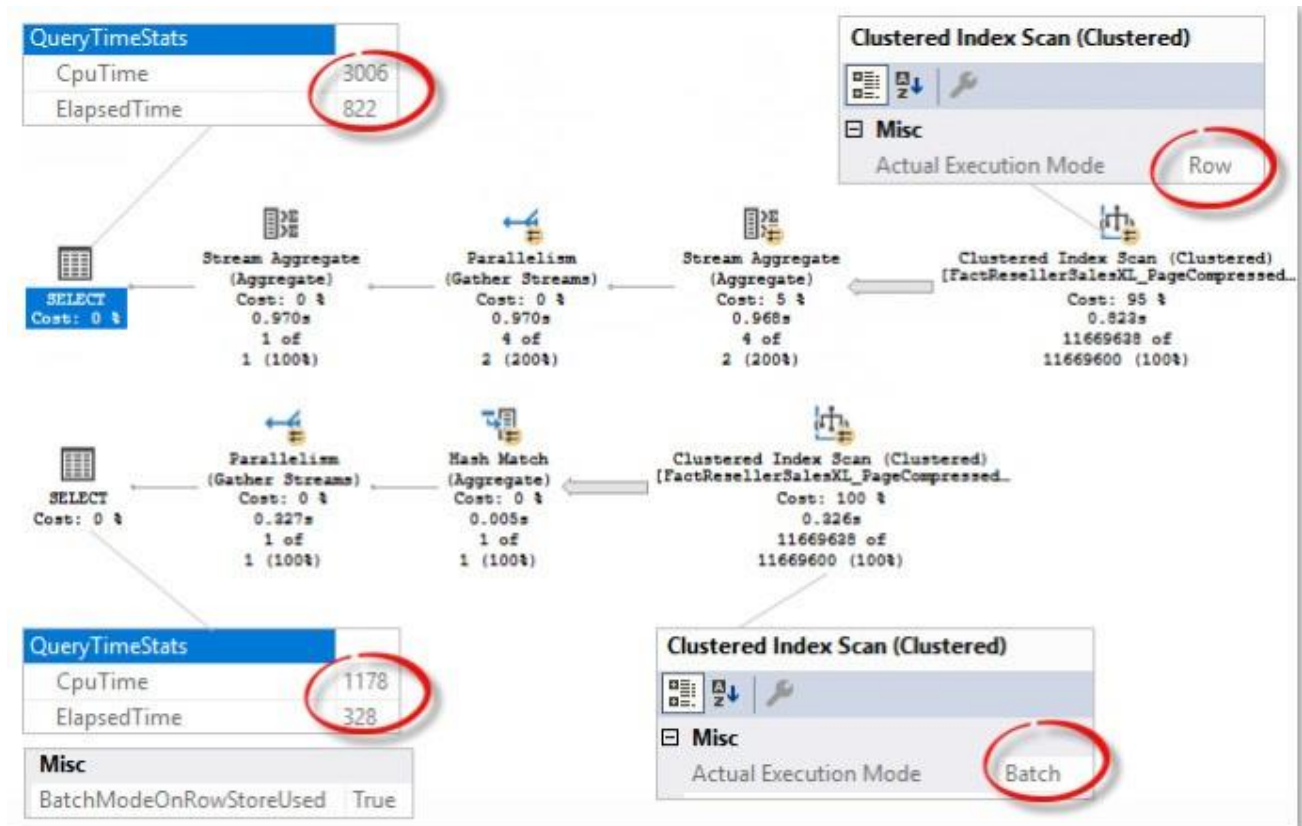
```
Select avg_fragmentation_in_percent, fragment_count from sys.dm_db_index_physical_stats
(db_id(N'SVU'), object_id('citible'), 1, null, 'limited');
```

```
Select avg_fragmentation_in_percent, fragment_count from sys.dm_db_index_physical_stats  
(db_id(N'SVU'), object_id('ncitable'), 2, null, 'limited');
```

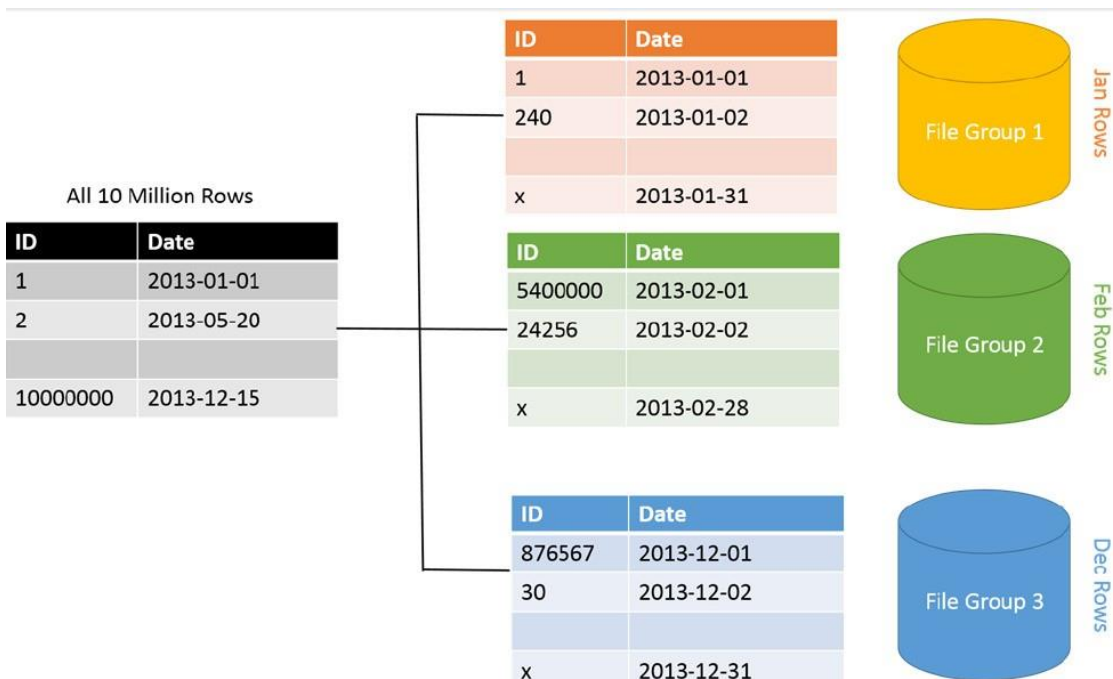
نرى أن نسبة ال Fragmentation أصبحت تقريبا = 0.00002 أو = 0 تماما

Batch-mode query processing is basically a vector-based query execution mechanism (processing in chunks of cache), which is tightly integrated with the Columnstore index. Queries that target a Columnstore index can use batch-mode to process up to 900 rows together, which enables efficient query execution, providing 3-4x in query performance improvement. In SQL Server, batch-mode processing is optimized for Columnstore indexes to take full advantage of their structure and in-memory capabilities.

To enable Batch Mode over Rowstore you should simply switch your DB to a latest compatibility level (CL), which is 150 in SQL Server 2019, the example shows the same query executed first as CL=140 (SQL Server 2017) and the second at 150 (2019):



2. Partitioning



SQL Server supports table and index partitioning. The data of partitioned tables and indexes is divided into units that can be spread across more than one filegroup in a database. The data is partitioned horizontally, so that groups of rows are mapped into individual partitions. All partitions of a single index or table must reside in the same database. The table or index is treated as a single logical entity when queries or updates are performed on the data.

2.1. **Benefits of Partitioning**

- فوائد ال Partitioning:

- (1) قفل Partition من ال Data عوضا عنها كلها عند القيام بأي عملية Maintenance مثل ال Rebuild ل Index
- (2) عند القيام بضغط (Compress) ل partition لا يتم تعطيل المستخدمين عن باقي ال Partitions
- (3) يتم زيادة سرعة أداء ال DB بسبب القراءة من كل Partition بشكل منفصل عوضا عن مسح ال DB كلها عند الإستعلام

- Ability to transfer or access subsets of data quickly and efficiently, while maintaining the integrity of a data collection. For example, an operation such as loading data from an OLTP to an OLAP system takes only seconds, instead of the minutes and hours the operation takes when the data is not partitioned.
- Ability to perform maintenance operations on one or more partitions more quickly. The operations are more efficient because they target only these data subsets, instead of the whole table. For example, you can choose to compress data in one or more partitions or rebuild one or more partitions of an index.
- Ability to improve query performance, based on the types of queries frequently run and on hardware configuration. For example, the query optimizer can process equi-join queries between two or more partitioned tables faster when the partitioning columns in the tables are the same, because the partitions themselves can be joined.
- When SQL Server performs data sorting for I/O operations, it sorts the data first by partition. SQL Server accesses one drive at a time, and this might reduce performance. To improve data sorting performance, stripe the data files of your partitions across more than one disk by setting up a RAID. In this way, although SQL Server still sorts data by partition, it can access all the drives of each partition at the same time.

- In addition, you can improve performance by enabling lock escalation at the partition level instead of a whole table. This can reduce lock contention on the table. To reduce lock contention by allowing lock escalation to the partition, set the LOCK_ESCALATION option of the ALTER TABLE statement to AUTO.

2.2. Components and Concepts

2.2.1. Partition function

هي Function خاصة بالتصنيف , يتم على أساسها التقسيم إلى مجالات حيث حين يتم إعطائها قيمة معينة , يعيد لنا في أي صنف موجودة

A database object that defines how the rows of a table or index are mapped to a set of partitions based on the values of certain column, called a partitioning column. That is, the partition function defines the number of partitions that the table will have and how the boundaries of the partitions are defined. For example, given a table that contains sales order data, you may want to partition the table into twelve (monthly) partitions based on a datetime column such as a sales date.

2.2.2. Partition scheme

هي أيضا Function خاصة بالتصنيف,

حيث حين يتم إعطائها قيمة معينة , تعيد لنا مكان الملف الموجودة فيه . عن طريق الربط بين ال Partition و ال Partition Function التي تعيد لكل قيمة أي Partition موجودة فيه

A database object that maps the partitions of a partition function to a set of filegroups. The primary reason for placing your partitions on separate filegroups is to make sure that you can independently perform backup operations on partitions. This is because you can perform backups on individual filegroups.

2.2.3. Partitioning column

- في حال كان ال Partitioned Column من نوع عامود محسوب (Computed Column) يتم تعليم هذا ال Column

ك Persisted (أي أن قيمته لن تتغير إلا إذا تغيرت قيم الحقول الداخلة في حساب قيمته) .

- هناك بعض أنواع الحقول التي لا يمكن تجزئتها مثل:

Ntext, text, image, xml, varchar(max), nvarchar(max), varbinary(max)

- مثال:

Use master

Go

Drop database if exists SVU

Go

Create database SVU

On (name='SVUdata', fname='E:\svu\bait\iis404\...\svu.mdf')

Log on (name='svulog', 'fname='E:\svu\bait\iis404\...\svu.ldf')

Go

بعد إنشاء ال DB, نقوم بإنشاء 3 File-Groups و لدينا ال Primary منشأ بشكل إفتراضي فيصبح لدينا 4 File Groups:

Alter database SVU add filegroup FG1

Alter database SVU add file (name='FG1file', filename='E:\svu\bait\iis404\...FG1file')

Alter database SVU add filegroup FG2

Alter database SVU add file (name='FG2file', filename='E:\svu\bait\iis404\...FG2file')

Alter database SVU add filegroup FG3

Alter database SVU add file (name='FG3file', filename='E:\svu\bait\iis404\...FG3file')

نقوم بإنشاء جدول ليس مجزء:

Use SVU

Create table notpartitionedtable

(id int identity (1,1) not null,

The date datetime not null,

Amount int)

ثم نقوم بتعبئته بمعلومات:

Declare @date datetime = '20170101'

While @date < getdate()

Begin

Set @date +=1

Insert into notpartitionedtable (the date, amount) values (@date, datediff(date

End

Declare @date1 datetime='20170101'

While @date1 < getdate()

Begin

Set @date1 +=1

Insert into notpartitionedtable (thedata, amount) values (@date1, -datediff(

End

Select count(id) from Notpartitionedtable

يظهر لدينا 3200 سجل تقريبا

ثم نقوم بإنشاء جدول غير مجزء و نقارن أدائه مع الجدول المجزء:

أولا, نضيف بيانات إضافية لجدول ال notpartitionedtable لأن بياناته ليست كثيرة

حيث نقوم بإعادة إضافة بياناته نفسها (نضيفها 10 مرات عن طريق إستخدام التعليمة Go 10)

Insert into notpartitionedtable (thedata, amount)

Select thedate, amount from notpartitionedtable

Go 10

نعيد الإستعلام عن عدد السجلات فيظهر لدينا 3 ملايين تقريبا :

Select count(id) from Notpartitionedtable

نقوم بتجربة Execute plan على الإستعلام التالي لنرى الأداء , تظهر قيمة ال cost = 11.64

Select thedate from notpartitionedtable

Where thedate between '20170101' and '20171231'

ننشأ Clustered Index على نفس الجدول الغير مجزء و نعيد إستخدام ال Execute plane لنرى الفرق

فتظهر لنا النتيجة cost = 3.07 :

Create clustered index clusteredindex

On notpartitionedtable (thedata)

Select thedate from notpartitionedtable

Where thedate between '20170101' and '20171231'

نقوم الآن بإنشاء جدول مجزء لنقارن أدائه مع أداء الجدول الغير مجزء السابق:

Partition Function - ونحدد نوع الحقل الذي سيمرر له بنوع Date

و نقوم بتمرير المجالات للسنوات من 2017 إلى 2019

حيث يصبح لدينا 4 مجالات: الأول قبل ال 2017 – الثاني من 2017 ل 2018 – الثالث من 2018 ل 2019 – الرابع من 2019 و بعد

Create partition function partitionfunction (date) as range (left)

For values ('20171231', '20181231', '20191231')

ثم ننشأ Partition Schema نقوم بأخذ الخرج الخاص بال Partition Function كما هو على الترتيب و وضعه في الملفات المحددة

FG1, FG2, FG3

Create partition schema partitionschema

As partition partitionfunction to ([primary], FG1, FG2, FG3)

نقوم الآن بإنشاء الجدول المجزء:

Create table partitionedtable (id int notnull, thedate date, amount int)

On partitionschema (thedata);

و ننشأ له Clustered Index:

Create clustered index clusteredindex

On partitionedtable (thedata) on partitionschema (thedata)

و نقوم بتعبئته ببيانات (ال 3 ملايين ريكورد الموجودين بال notpartitionedtable) لنقوم بالبدا بالمقارنة:

Insert into partitionedtable select * from notpartitionedtable

نقوم بالمقارنة بالأداء بين الجدولين.

نحصل على نتيجة $cost = 3.07$ لجدول ال notpartitionedtable

و قيمة $cost = 2.6$ لجدول ال partitionedtable

Select thedate from notpartitionedtable where thedate between '20170101' and

Select thedate from partitionedtable where thedate between '20170101' and

Select index_id, partition_number, rows from sys.partitions where object_id=

- نستطيع تقسيم ال partition بحال أصبح كبير جدا عن طريق تقسيم ال Range الذي أصبح كبير جدا:

Alter partition function [artitionfunction () split range ('20180331')

- لغضط Partition:

نقوم أولا بإختبار إذا عملية الضغط سنتسفيد منها ...

و ذلك من خلال تطبيق Procedure على ال Partition تقوم بحساب حجمه بعد الضغط و مقارنته مع قبل الضغط

فإذا وجدنا أن الحجم إختلف كثيرا عند ضغطه, نقوم بعملية الضغط. و إذا لم يختلف الحجم جدا لا نقوم بها.

Exec sp_estimate_data_compression_saving @shcema_name='dbo',


```
Object_name = partitionedtable, @index_id = null, @partition_number = null,  
@date_compression = 'ROW'
```

بعد تجربة ال Prcedure نرى أن الحجم قد اختلف تقريبا الضعف, لذلك نقوم بضغط باستخدام ال Rebuild:

```
Alter table partitionedtable rebuild partition = 3 with ( data_compression = ROW )
```

بعد الإنتهاء من عملية الضغط, نقوم بإعادة تجربة ال procedure فنرى أن الحجم لن يتغير في حال القيام بضغط جديد:

```
Exec sp_estimate_date_compression_saving @shcema_name='dbo',  
Object_name = partitionedtable, @index_id = null, @partition_number = null,  
@date_compression = 'ROW'
```

The column of a table or index that a partition function uses to partition the table or index. Computed columns that participate in a partition function must be explicitly marked PERSISTED.

All data types that are valid for use as index columns can be used as a partitioning column, except timestamp. The ntext, text, image, xml, varchar(max), nvarchar(max), or varbinary(max) data types cannot be specified. Also, Microsoft .NET Framework common language runtime (CLR) user- defined type and alias data type columns cannot be specified.

2.3. Partitioning Steps

- Create a filegroup or filegroups and corresponding files that will hold the partitions specified by the partition scheme.
- Create a partition function that maps the rows of a table or index into partitions based on the values of a specified column.

- Create a partition scheme that maps the partitions of a partitioned table or index to the new filegroups.
- Create or modify a table or index and specify the partition scheme as the storage location.

3. Full-text Search

- و تدعى هذه الطريقة أيضا ب **Full-text Index**
- تتقدم هذه الطريقة سرعة كبيرة عند البحث في الحقول من نوع نص, مثل Char ,text, varchar, ... حيث تقدم القدرة على البحث على كافة حالات الكلمة أو الفعل المراد البحث عنهم مثل: عند البحث عن كلمة **Man** سيقوم بالعثور أيضا على كلمة **Men** و عند البحث عن فعل **Fall** سيقوم بالعثور أيضا على فعل **Fell, Fallen** .
- نرى فاعلية هذه الطريقة عند وجود ملايين السجلات

- لإستخدام هذه النوع من ال **Index** نقوم ب :
- (1) إنشاء **Full-text Catalog**
- (2) إنشاء **full-text index** على الجداول أو ال **Indexed View** الذي نريد البحث ضمنها

- صيغة إستعمال ال **Full-text Index** :
- (1) في حال البحث عن مجموعة كلمات أو عبارات:

Select comments

From productuin.ProductReview

Where CONTAINS (comments, ‘ “learning curve” ‘)

- (2) في حال البحث عن مجموعة كلمات أو عبارات تبدأ بحرف معين:

Select Description, ProductionDescriptionID

From Productuin.ProductionDescription

Where CONTAINS (Description, ‘ “top*” ‘)

- (3) في حال البحث عن كافة حالات عبارة أو فعل معين:

Select comments, ReviewerName

From Production.ProductionReview

Where CONTAINS (comments, 'FORMSOF (INFELECTIONAL, 'foot'))

حيث سيتم البحث عن جميع جالات كلمة foot

- ممكن البحث عن كلمة بقرب كلمة حيث نستخدم الخيار near

Full-Text Search in SQL Server and Azure SQL Database lets users and applications run full-text queries against character-based data in SQL Server tables. A full-text index includes one or more character-based columns in a table. These columns can have any of the following data types: char, varchar, nchar, nvarchar, text, ntext, image, xml, or varbinary(max) and FILESTREAM.

Each full-text index indexes one or more columns from the table, and each column can use a specific language.

Full-text queries perform linguistic searches against text data in full-text indexes by operating on words and phrases based on the rules of a particular language such as English or Japanese. Full-text queries can include simple words and phrases or multiple forms of a word or phrase. A full-text query returns any documents that contain at least one match (also known as a *hit*). A match occurs when a target document contains all the terms specified in the full-text query, and meets any other search conditions, such as the distance between the matching terms.

To support full-text queries, full-text indexes must be implemented on the columns referenced in the query. There are two basic steps to set up full-text search

- Create a full-text catalog.
- Create a full-text index on tables or indexed view you want to search.

3.1. Full-text Queries

After columns have been added to a full-text index, users and applications can run full-text queries on the text in the columns. These queries can search for any of the following:

- One or more specific words or phrases (simple term)

```
SELECT Comments
FROM Production.ProductReview
WHERE CONTAINS(Comments, '"learning curve"')
```

- A word or a phrase where the words begin with specified text (prefix term)

```
SELECT Description, ProductDescriptionID
FROM Production.ProductDescription
WHERE CONTAINS (Description, '"top*"' )
```

- Inflectional forms of a specific word (generation term)

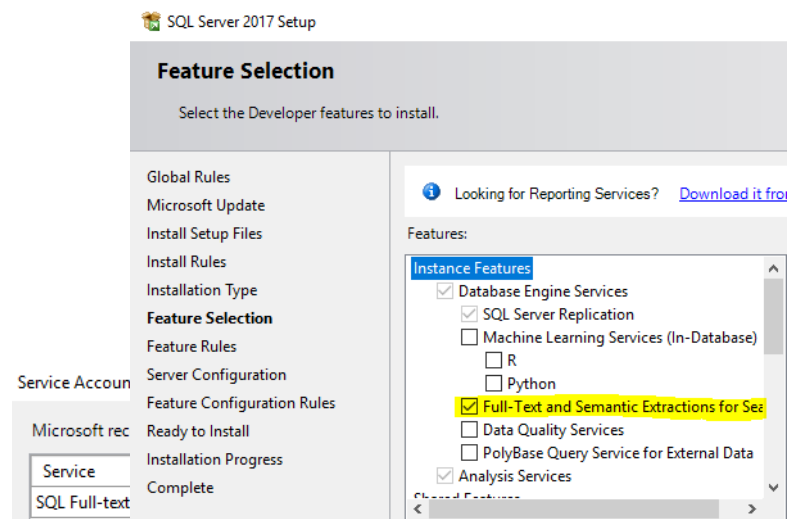
```
SELECT Comments, ReviewerName
FROM Production.ProductReview
WHERE CONTAINS (Comments, 'FORMSOF(INFLECTIONAL, "foot")')
```

- A word or phrase close to another word or phrase (proximity term) by using “Near”
- Synonymous forms of a specific word (thesaurus)
- Words or phrases using weighted values (weighted term)

Example: *Select id From products Where Contains (Description, "Snap Happy 100EZ" Or FORMSOF(THESAURUS, 'Snap Happy') OR '100EZ') AND Cost < 200 ;*

3.1.1. Set up full-text search steps

I. Enable full text at server level



II. Enable full text at database level: `Exec sp_fulltext_database 'enable';`

III. Create a full-text catalog: Create Fulltext Catalog textCatalog with accent_sensitivity = off

IV. Create a full-text index on tables or indexed view you want to search.

Create Fulltext Index on myTable(The textColumn) Key Index pkId on textCatalog with Stoplist=Off

3.1.2. Full-Text Search Vs LIKE

- كل من **Like, Full-text search** له استخدام أفضل حسب القيمة المراد البحث عنها

- عندما نريد البحث عن قيمة **Binary** فإن استخدام **like** أفضل

- عندما نريد البحث عن قيمة **text** فإن إستخدام **Full-text search** أفضل

- في حال لدينا ملايين السجلات فإن إستخدام **Full-text Search** أفضل

- في حال نريد البحث عن جميع أشكال الفعل أو الكلمة فإن إستخدام **Full-text Search** أفضل

- LIKE predicate works on character patterns only.
- You cannot use the LIKE predicate to query formatted binary data.

- A LIKE query against millions of rows of text data can take minutes to return; whereas a full- text query can take only seconds or less against the same data, depending on the number of rows that are returned.

4. Buffer Pool Extension

- نذكر أن ال **Column Store Index** يضغط البيانات و يعطي أداء أسرع ب 10 مرّات لأنه يضع ال **Data** في الذاكرة

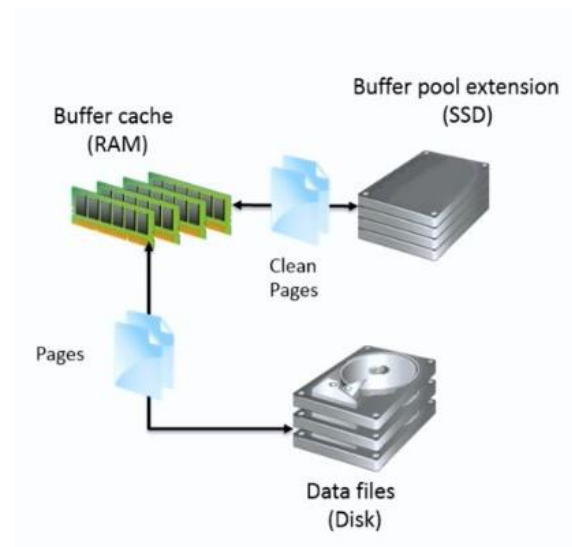
و يعتبر أفضل إستخدام له في تحليل البيانات مع التتابع التجميعية مثلاً

- و بالنسبة لقواعد البيانات من نوع **OLTP** فلدينا **SQL Server in-memory OLTP engine**

الذي يضع ال **Data** أيضاً في الذاكرة و في حال حدودية الذاكرة لدينا, نستخدم خاصية ال **Buffer Extension** حيث نستخدم من مساحة الهارد ال **SSD** (حصراً **SSD**) و نحولها إلى **Ram**

The buffer pool extension provides the seamless integration of a nonvolatile random access memory (that is, solid-state drive) extension to the Database Engine buffer pool to significantly improve I/O throughput.

The following list describes the benefits of the buffer pool extension feature.



- لتفعيل خاصية ال Buffer Pool Extension:

نستخدم التعليمة: (حيث قمنا بتحديد القرص الذي سنأخذ منه مساحة, مع تحديد حجم المساحة المراد)

Alter server configuration set buffer pool extension on (Filename='E:\mycash.bpe' size=32GB)

و للتحقق من معلومات ال Buffer Pool:

Select * from sys.dm_os_buffer_pool_extension_configuration

أو بإستخدام التعليمة التالية:

Select * from sys.dm_os_buffer_descriptors

و لإيقاف ال Buffer Pool Extension:

Alter server configuration set buffer pool extension off

- من فوائد إستعمال خاصيّة Buffer Pool Extension:

- Increased random I/O throughput
- Reduced I/O latency
- Increased transaction throughput
- Improved read performance with a larger hybrid buffer pool
- A caching architecture that can take advantage of present and future low-cost memory drives

4.1. Terms and concepts

Solid-state drive (SSD): Solid-state drives store data in memory (RAM) in a persistent manner.

Buffer: In SQL Server, A buffer is an 8-KB page in memory, the same size as a data or index page.

Buffer pool: Also called buffer cache. The buffer pool is a global resource shared by all databases for their cached data pages.

Checkpoint: A checkpoint creates a known good point from which the Database Engine can start applying changes contained in the transaction log during recovery after an unexpected shutdown or crash. A checkpoint writes the dirty pages and transaction log information from memory to disk and, also, records information about the transaction log.

5. Memory optimized table

- يستخدم مع ال OLTP – Online Transaction Processing (البرامج التي تجوي قراءة و كتابة بشكل متواصل من قاعدة البيانات مثل البرامج المحاسبية) حيث بعد إنشائه يتم عمل compile له من قبل ال system و تحويله لنوع dll ثم نقله للذاكرة
- أهم الفروقات بين Memory optimized-table و ال disk-based table هي أنه يتم تخزين ال data في ال Memory optimized table على شكل أسطر, وليس على شكل صفحات كما في ال disk-based table
- طالما البيانات في الذاكرة فلا يتم التشبيك عليها (لا تحتوي على Checksum, و لا يتم نقل الصفحة إلى ال log و الكتابة عليها لتصبح dirty page , ولا يمكن القيام بضرب لأي صفحة)
- يحتاج ال memory optimized table ل file group خاصة به
- Checkpoint : (سابقا :) هي العملية التي بها الكتابة على ال file
- بينما مع ال memory optimized-table أصبح إسمها Checkpoint file و أصبح تعاملها مختلف مع الأسطر و عملية الكتابة
- مثال عن ال checkpoint file في ال memory optimized table:
- في حال لدينا عدة أسطر, و أردنا التعديل على سطر معين..
- سيتم حذف هذا السطر و وضعه في delta, بعد ذلك يتم إضافة سطر جديد (يمثل التعديل على القديم)
- في كل memory optimized table يوجد: data, delta
- و يقوم بعمل Row visioning للتفريق بينهم
- تتيح ال Checkpoint files بالإضافة لل transactions log files (ملفات ال data, delta) خاصية ال Full Durability (يعني عمر أطول)

The SQL Server in-memory OLTP engine allows you to create in-memory optimized OLTP tables within your existing relational database. “Memory-optimized” tables (as opposed to standard, “disk-based” tables) reside completely in-memory. A key difference of memory-optimized tables over disk-based tables is that memory-optimized tables store the data as rows. No pages need to be read into memory when the memory-optimized tables are accessed.

A set of checkpoint files (data and delta file pairs) is created in a memory-optimized file group for data persistence, similar to the data files used for disk-based tables. However, these checkpoint files, unlike data files used for disk-based tables, are append-only, allowing SQL Server to leverage the full I/O bandwidth of the storage. These checkpoint files, along with the transaction log, provide full durability and are used for recovery at restart or for database backup/restore.

There are two main types of in-memory-optimized OLTP tables: **SCHEMA_AND_DATA** (i.e. durable tables) and **SCHEMA_ONLY** (i.e. non-durable tables). The first type provides full durability guarantee just like disk-based tables for your OLTP workload. It is the default setting when creating memory-optimized tables. The second type persists the schema of the table but not the data. **SCHEMA_ONLY** would be used in scenarios where OLTP workloads do not require data persistence, such as session state management for an application, for staging tables in an ETL scenario, or as a replacement for temporary tables in TempDB that use a table type.

- Defined as C structs, compiled into DLLs, and loaded into memory
- Can be persisted as filestreams, or non-durable
- Do not apply any locking semantics
- Can be indexed using hash indexes
- Can co-exist with disk-based tables
- Can be queried using Transact-SQL
- Cannot include some data types, including text, image, and nvarchar(max)
- Do not support identity columns or foreign key constraints

- هناك نوعين أساسيين من ال memory-optimized table :
(durable tables) :Schema_and_data (1)

توفّر عمر أطول للبيانات (و لا يتم ضياعها عند القيام ب restart لل DB مثلا) كما في ال disk-based tables , و تكون الخيار النوع الافتراضي عند إنشاء memory-optimized table

(non-durable tables) :Schema_only (2)

- يحافظ على ال Schema الخاصة بال table وليس على ال Data التي بداخله (عند القيام ب restart لل DB ستضيع ال Data ولكن ستبقى الجداول)

- يستخدم كبديل لل Temp table الذي أيضا يتم ضياعه عند القيام ب restart لل DB و لكن ال schema only يكون أسرع بكثير لأنه محفوظ على الذاكرة

- يستخدم أيضا في عملية ال Staging.

حيث إن كان لدينا OLTP , DataWH , فإن عملية نقل البيانات من ال OLTP إلى ال DataWH تدعى

ETL - Extract Transform Load ليتيم معالجتها و الحصول على معلومة نافعة منها كنتيجة

و عند وجود عدة مصادر لل OLTP تحتوي معلومات منسقة بشكل مختلف , يتم عمل جدول واحد يحوي كل هذه المعلومات بشكل واحد و تدعى هذه العملية ب Staging

- خلاصة معلومات عن ال memory-optimized table :

➤ تعرّف كملفات من لغة C , يتم عمل compile لها لملفات DLL , و تحفظ في الذاكرة

➤ ممكن أن تكون non-Durable , أو filestreams (durable)

➤ لا يمكن إستخدامها لقلل أي معلومة

- يمكن إنشاء فهرس لها من نوع (hash, range) non-clustered indexes
- يمكن أن تتواجد مع ال disk-based tables
- لا يدعم وجود علاقة بينه وبين disk-table , ولا يمكن أن يحتوي على FK من disk table
- يمكن القراءة منه باستخدام T-SQL
- لا يمكن أن يحتوي على بعض أنواع الحقول مثل Text, image, Binary

- اسباب سرعة ال memory-optimized table
- عدم وجود أقفال
- يستخدم ال hash index, range index

5.1. **Indexes on memory-optimized tables**

- ذكرنا أنه ممكن استخدام نوعين من ال non-clustered index مع ال memory-optimized table
- Hash index يناسب ال Exact match
- Range Index يناسب ال Range Values
- و لا داعي للقيام بصيانة لهما, حيث عند كل عملية restart يتم تطبيق صيانة لهما بشكل أوتوماتيكي

Memory-optimized tables support two types of nonclustered indexes: hash and range indexes. Hash indexes provide optimal access paths for equality searches, while range indexes are used for queries involving range predicates or for ordered retrieval of the data.

Every memory-optimized table must have at least one index. For durable memory-optimized tables, a unique index is required to uniquely identify a row when processing transaction log records change during recovery. Indexes on in-memory tables reside only in-memory. They are not stored in checkpoint files nor are any changes to the indexes logged. The indexes are maintained automatically during all modification operations on memory-optimized tables, just like B-tree indexes on disk-based tables, but if SQL Server restarts, the indexes on the memory-optimized tables are rebuilt as the data are streamed into memory.

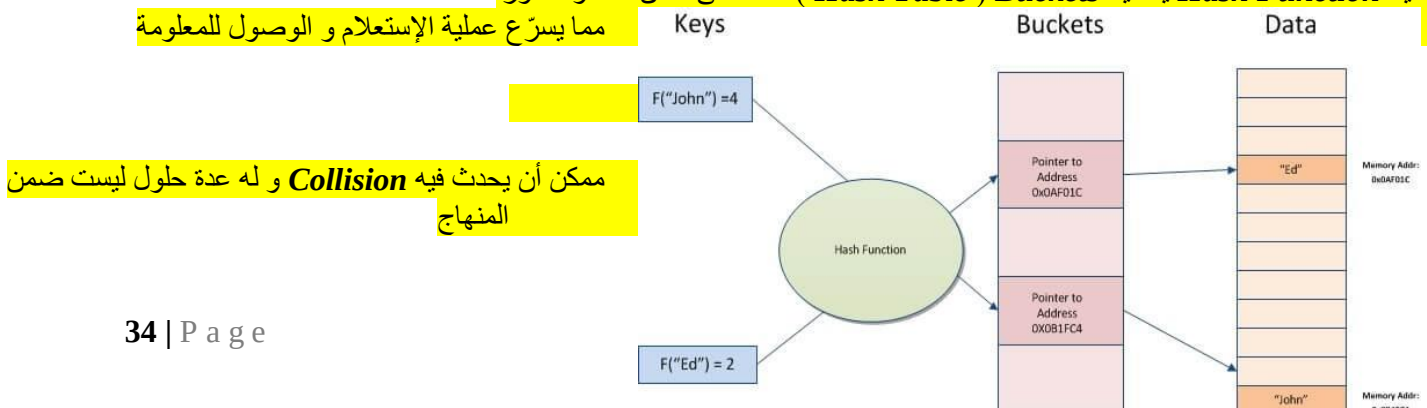
5.2. Concurrency Improvements

- توجد في الذاكرة، ليس عليه أي أقفال، و تعتمد في عملها على مبدأ **optimistic concurrency control** الذي لا يفرض أقفال أو **latches** (من باب التفاؤل بأنه لن يحدث أي تعارض)
- يتم تطبيق مبدأ عمل ال **Optimistic Concurrency Control** باستخدام ال **Row Versions** التي تحفظ في الذاكرة على عكس ال **disk-based tables**
- ال **Latches** هي آلية يستخدمها ال **SQL Server** لحماية الذاكرة من التعارضات (على مستوى ال **page**)

Applications whose performance is affected by engine-level concurrency, such as latch contention or blocking, improve significantly when the application is migrated to in-memory OLTP. Memory-optimized tables do not have pages, so there are no latches and hence no latch wait. If your database application encounters blocking issues between read and write operations, in-memory OLTP removes the blocking issues because it uses optimistic concurrency control to access data. The optimistic control is implemented using row versions, but unlike disk-based tables, the row versions are kept in-memory. Since data for memory-optimized tables are always in-memory, the waits due to I/O path are eliminated. Also, there will be no waits for reading data from disks and no waits for locks on data rows.

5.3. Memory optimized table – Hash Index

- للوصول لمعلومة في جدول معين من نوع **memory optimized table** الذي يستخدم **Hash Index** فإن لدينا **Hash Function** يعطينا **Buckets (Hash Table)** تدلنا على مكان المعلومة فوراً مما يسرع عملية الاستعلام و الوصول للمعلومة



ممكن أن يحدث فيه **Collision** و له عدة حلول ليست ضمن المنهاج

What is a hash index?

An array of N buckets or slots, each one containing a pointer to a row. Hash indexes use a hash function $F(K, N)$ in which given a key K and the number of buckets N, the function maps the key to the corresponding bucket of the hash index. Buckets only store the memory address in which the data is placed.

What is a hash function?

A hash function is any algorithm that maps data of variable length to data of a fixed length in a deterministic and close to random way. A very

simple hash function would be a string that returns its length, so $F(\text{"John"}) = 4$ and $F(\text{"Ed"}) = 2$. If we define a hash index of 5 buckets using this function, then the pointers that point to "John" and "Ed" are stored at buckets 4 and 2 respectively. The following image will help you understand.

5.4. Creating Memory optimized table

- Add a filegroup for memory-optimized data

```
ALTER DATABASE MyDB
ADD FILEGROUP mem_data CONTAINS MEMORY_OPTIMIZED_DATA;
GO
ALTER DATABASE MyDB
ADD FILE (NAME = 'MemData' FILENAME =
'D:\Data\MyDB_MemData.ndf')
TO FILEGROUP mem_data;
```

- Create a memory-optimized table

```
CREATE TABLE dbo.MemoryTable
(OrderId INTEGER NOT NULL PRIMARY KEY NONCLUSTERED HASH WITH (BUCKET_COUNT = 1000),
OrderDate DATETIME NOT NULL,
ProductCode INTEGER NULL,
Quantity INTEGER NULL)
WITH (MEMORY_OPTIMIZED = ON, DURABILITY = SCHEMA_AND_DATA);
```

5.4.1. Planning data migration to memory-optimized tables

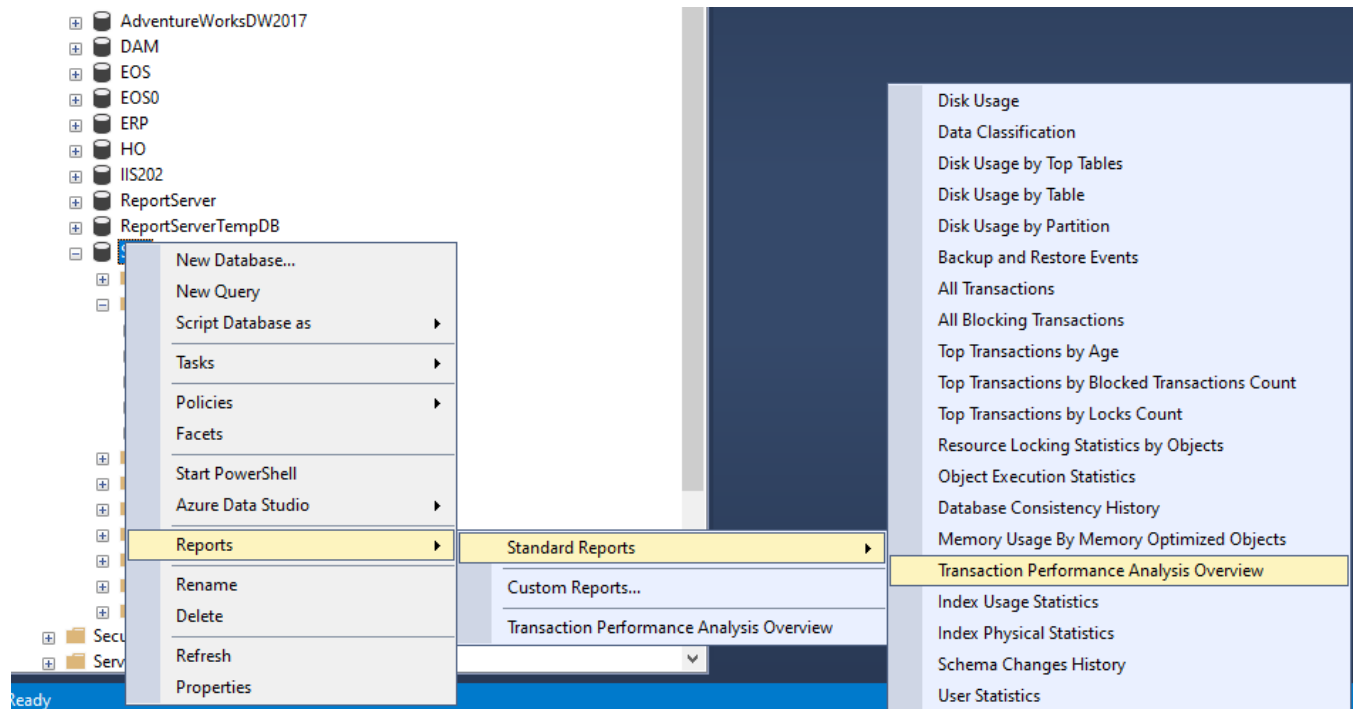
- التخطيط للتقرير إذا سنفيد من تحويل table إلى memory-optimized table أم لا:

Microsoft **SQL Server Management Studio (SSMS)** contains tools to help analyze and migrate tables to memory-optimized storage.

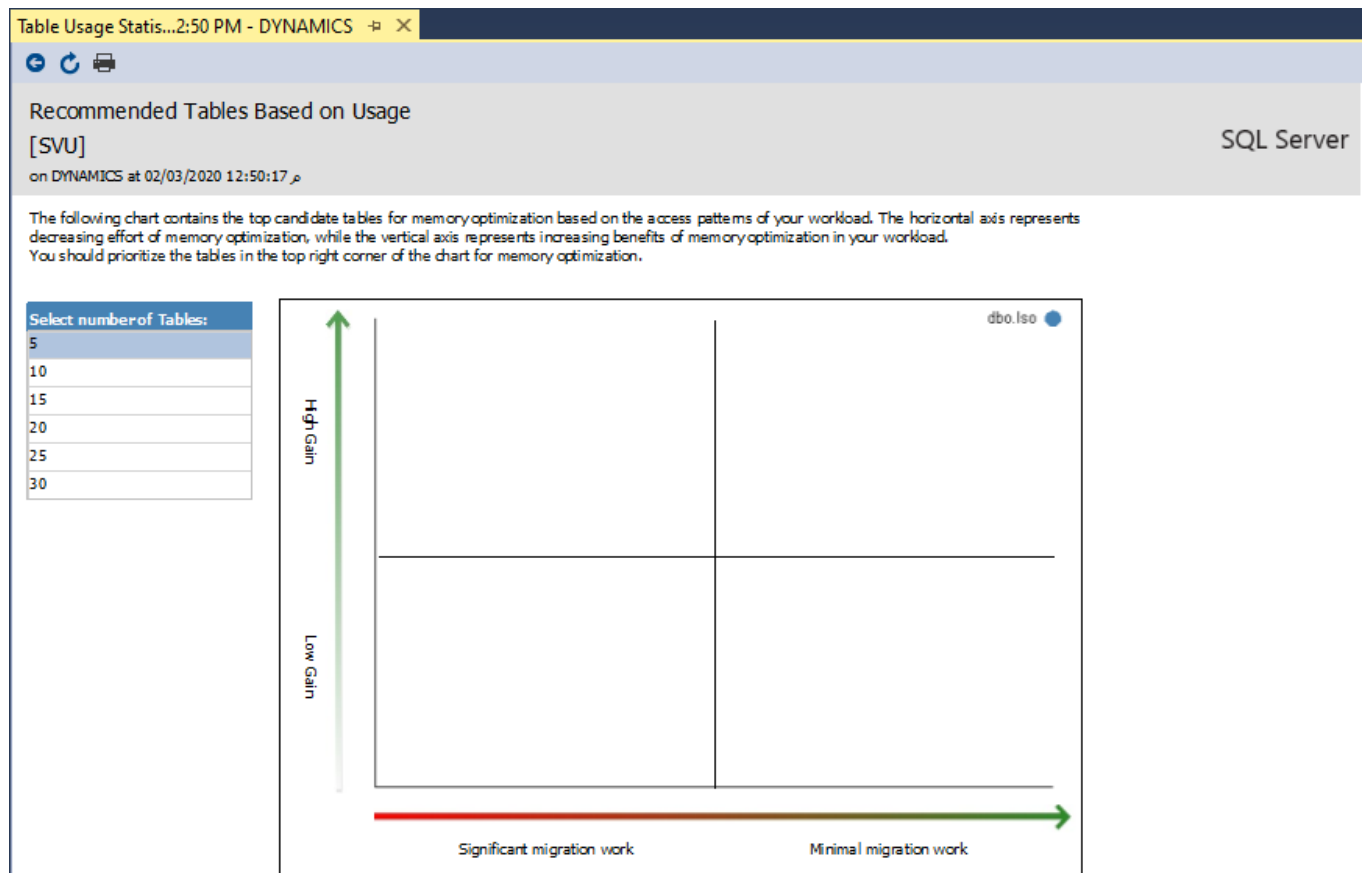
When you right-click on a database in SSMS and click on **Reports | Standard Reports |**

Transaction Performance Analysis Overview, a four-quadrant report of all tables in the database will be made:

Transaction Performance analysis overview نوقف على ال DB و نضغط باليمين، و نختار حيث سيتم عرض جميع الجداول التي تحتوي مناقلات من هذه ال DB ، مع تحديد نسبة الإستفادة من تحويلهم لل memory-optimized table و كمية المعلومات التي بادخلهم



The report will look at each table and place it on the chart to show the ease of migration versus the expected gain by migrating the table to be memory-optimized:

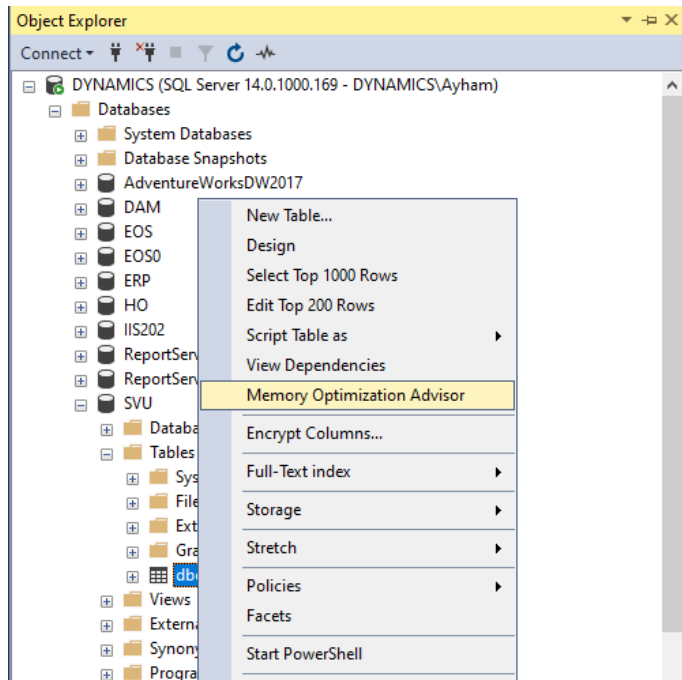


و بعد تحديد الجدول الذي نريد تحويله, نضغط عليه باليمين و نختار memory optimization advisor

حيث سيقوم بإعطاء تحذير في حال وجود FK في هذا الجدول مرتبطة مع جداول أخرى من نوع disk-based tables

حيث سيتم إلغائهم (لا يحتوي ال memory-optimized table على FK مرتبطة مع جداول من نوع disk-based tables)

Once you have identified tables that might benefit, you can right-click on individual tables and run the Memory Optimization Advisor:



The **Table Memory Optimization Advisor** is a "wizard" style of user interface that will step you through the configurations:

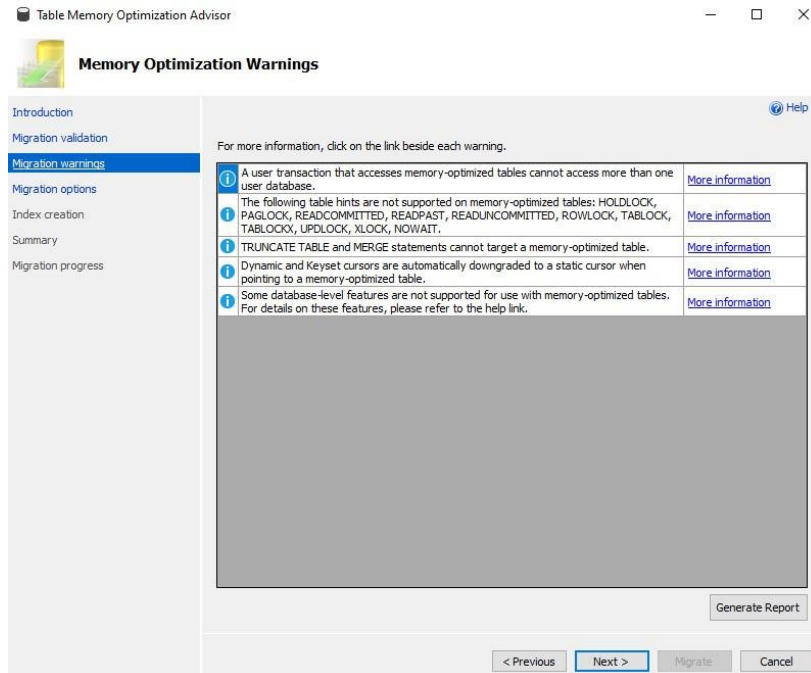
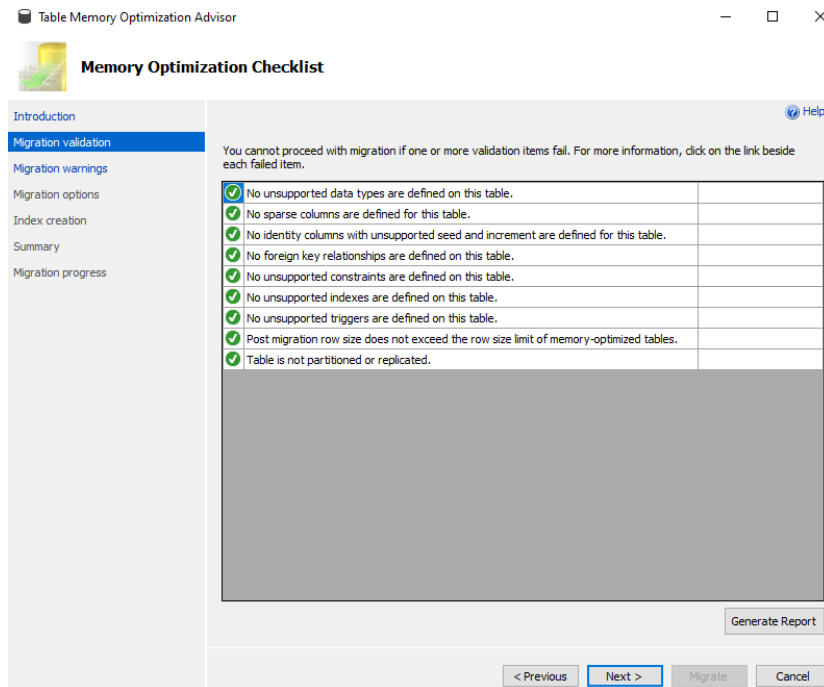


Table Memory Optimization Advisor

Review Optimization Options

Introduction
Migration validation
Migration warnings
Migration options
Index creation
Summary
Migration progress

Specify options for memory optimization:

Memory-optimized filegroup:

Logical file name:

File path:

Rename the original table as:

Estimated current memory cost (MB):

☐ Also copy table data to the new memory optimized table.

By default, this table will be migrated to a memory-optimized table with both schema and data durability.

☒ Check this box to migrate this table to a memory-optimized table with no data durability.

< Previous **Next >** Migrate Cancel

But if you a memory optimized filedroup does not exist, then the wizard will create the filegroup with a file.

Table Memory Optimization Advisor

Create Index

Introduction
Migration validation
Migration warnings
Migration options
Index creation
Summary
Migration progress

A memory-optimized table requires at least one index. Choose the columns you wish to include in the index of the table:

Column	Type
☒ id	int
☐ Description	varchar

Select a new name for this index: ☐ Is a primary key

Select the type of this index:

☒ Use NONCLUSTERED HASH index
A NONCLUSTERED HASH index provides the most benefit for point lookups. It provides no discernible benefit if a query is running a Range scan.
Bucket Count:
The Bucket Count of a NONCLUSTERED HASH index is the number of buckets in the hash table. It is recommended to set the fill factor to 50 to 60% if the table requires a lot of space for growth. Bucket Count will be rounded up to the nearest power of two.

☐ Use NONCLUSTERED index
A NONCLUSTERED index provides the most benefit for range predicates and ORDER BY clauses. NONCLUSTERED indexes are unidirectional. It provides no benefit for ORDER BY clauses with orders different from the index.
Sort column and order:

Column	Sort Order
--------	------------

< Previous **Next >** Migrate Cancel

5.5. Natively compiled stored procedures

- يتم أيضا عمل **compile** لها لتصبح ملفات من نوع **DLL** ثم تنتقل إلى الذاكرة
- تعمل فقط مع الجداول من نوع **memory-optimized table**

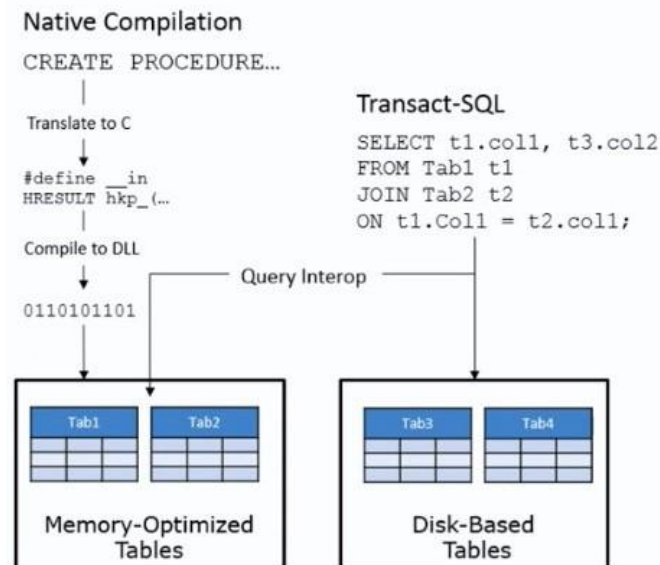
SQL Server can natively compile stored procedures that access memory-optimized tables. A natively

compiled stored procedure optimizes TSQL statements, transforms them into Visual C code, and

then generates a DLL. This enables SQL Server to execute the business logic in the stored procedure at an order of magnitude of better efficiency using fewer instructions compared to traditional stored procedures. SQL 14 allows commonly used TSQL statements in OLTP workloads to be used inside natively stored procedures. The SQL Server team continues to expand the TSQL surface area in new releases.

5.6. Querying Memory optimized table

- Query Interop
 - Interpreted Transact-SQL
 - Enables queries that combine memory-optimized and disk-based tables
- Native Compilation
 - Stored procedure converted to C and compiled
 - Access to memory-optimized tables only



- نقوم بالإستعلام داخل ال memory optimized table باستخدام أحد النوعين من ال Query:

(1 Query Interop: مثل ال T-SQL Query (ال Query العادية)

(2 Native Compilation: التي تكون عبارة عن Stored Procedure

5.6.1. Creating Native Stored Procedure

- لإنشاء Native Stored Procedure نستخدم ال script التالي في الصورة :

- Use the CREATE PROCEDURE statement
 - NATIVE_COMPILATION option
 - SCHEMABINDING option
 - EXECUTE AS option
 - BEGIN ATOMIC clause (isolation level and language)

```
CREATE PROCEDURE dbo.DeleteCustomer @CustomerID INT
WITH NATIVE_COMPILATION, SCHEMABINDING, EXECUTE AS OWNER
AS
BEGIN ATOMIC WITH
    (TRANSACTION ISOLATION LEVEL = SNAPSHOT;
    LANGUAGE = 'us_English')
DELETE dbo.OpenOrders WHERE CustomerID = @CustomerID
DELETE dbo.Customer WHERE CustomerID = @CustomerID
END;
```

- مثال عن استخدام ال Memory-Optimized data:

```
Use master
Go
Drop database if exists SVUO
```

```
Create database SVUO
On (name='svudata' , filename='E:\svu\bait\.....\svu.mdf')
Log on (name='svulog' , filename='E:\svu\bait\.....\svu.ldf')
```

نقوم بإنشاء filegroup اسمه mod و نعين أنه من نوع memory-optimized data
Alter database SVUO add filegroup fg_mod contains memory_optimized_data

ثم نضيف ملف لل filegroup الجديد

```
Alter database SVUO add file (name='mod1' , Filename='E:\svu\bait\....\mod1.ndf') to filegroup fg_mod
```

```
Alter database SVUO set memory_optimized_elevant_to_snapshot = on
```

نقوم بإنشاء durable memory optimized table:

```
Use SVUO
```

نقوم بإنشاء الجدول Employee (مفتاحه الرئيسي العامود id و مطبق عليه nonclustered hash حيث قيمة ال bucket_count = 1000000) مع تحديد استخدام hash index و تفعيل ال memory_optimized و تحديد نوع ال durability ب schema_and_data (في حال لم نحدد نوع ال durability فأیضا سيتم استخدام النوع schema_and_data لأنه الافتراضي)

```
Create table memorytable
( order_id int primary key nonclustered hash with (bucket_count=1000000),
Order_date date notnull, )
With ( memory_optimized = on, durability = schema_and_data )
```

نقوم بإنشاء جدول من نوع disk-table (عادي) مشابه للجدول السابق:

```
Create table disktable
( order_id int primary key nonclustered, order_date date notnull
```

نضيف بعض البيانات لجدول disktable: (نضيف 500 الف record)

```
Begin tran
Declare @theid int=0
While @theid < 500000
Begin
Set @theid = @theid + 1
Insert into disktable values ( @theid, getdate () )
End
Commit
```

يأخذ معنا التنفيذ لتعبئة الجدول زمن 28 ثانية , و للتأكد من عدد البيانات في جدول disktable:

```
Select count(*) from disktable
```

ثم نقوم بتعبئة جدول ال optimized-data بنفس عدد ال records (500 الف record) لنرى كمقارنة كم سيحتاج لوقت:

```
Begin tran
Declare @theid int=0
While @theid < 500000
Begin
Set @theid = @theid + 1
Insert into memorytable values ( @theid, getdate () )
End
Commit
```

يأخذ معنا التنفيذ لتعبئة الجدول زمن 5 ثواني فقط

نقوم بتجربة التعبئة باستخدام ال native stored procedure : (مع تفعيل ال schema binding و التي تعني أن جميع ال objects المرتبطة بهذه ال procedure لا يمكن حذفها أو التعديل عليها)

نقوم أولاً بتعريف ال procedure :

```
Go
Create proc usp_insertintomem with native_compilation, schemabinding, exec
as
Begin atomic with ( transaction isolation level = snapshot, language = 'us_
Declare @theid int=0
While @theid < 500000
Begin
Set @theid = @theid + 1
Insert into dbo.memorytable values ( @theid, getdate () )
End,
Return;
End;
```

Select count(*) from memotytable

Dbcc freesystemcash ('all')

نقوم بتنفيذ ال procedure :

Exec usp_insertintomem

نرى أنه تم تنفيذها خلال 0 ثانية

5.7. *In-memory OLTP customer success story*



bwin.party was formed from a merger between two gaming giants, bwin Interactive Entertainment and PartyGaming, each of which had high-traffic gaming websites. Consolidation of their websites resulted in huge overloads on their infrastructure. bwin.party needed to overcome its scalability issues and wanted to improve the performance of its gaming website to support rapid business growth.

Solution: Using the Microsoft in-memory OLTP solution in SQL Server 2014, their gaming systems can now handle 250,000 requests per second (almost 20 times the original load) and offer players a faster, smoother gaming experience.

References

de Bruijn. J et. al., "SQL Server In-Memory OLTP and Columnstore Feature Comparison", Microsoft Press (2016).

LeBlanc. P, Assaf. W, Jackson. B, Curnutt M; "SQL Server 2017 Administration Inside Out", Microsoft Press (2018).

Microsoft SQL Server Mission-Critical Performance, **Technical White Paper**, Microsoft (2016)

Gorman. K, et al., "Introducing Microsoft SQL Server 2019, Reliability, scalability, and security both on premises and in the cloud", Packt Publishing (2019)

<https://docs.microsoft.com/en-us/sql/relational-databases/in-memory-oltp/introduction-to-memory-optimized-tables?view=sql-server-ver15>

<https://docs.microsoft.com/en-us/sql/relational-databases/indexes/clustered-and-nonclustered-indexes-described?view=sql-server-ver15>

<https://docs.microsoft.com/en-us/sql/relational-databases/partitions/partitioned-tables-and-indexes?view=sql-server-ver15>

<https://docs.microsoft.com/en-us/sql/relational-databases/indexes/columnstore-indexes-overview?view=sql-server-ver15>

<https://docs.microsoft.com/en-us/sql/relational-databases/search/full-text-search?view=sql-server-ver15>